MANUGOM STOCK

Projet personnel de développement web

Application Web de Gestion de Stock et de Commandes de Pneus

Rapport de conception, de réalisation et de validation

Réalisé par : Youssef Bouikourar

Remerciements

Je remercie les personnes qui ont échangé avec moi sur ce projet et partagé leurs remarques. Ces échanges m’ont aidé à préciser les besoins, à vérifier les parcours et à améliorer la présentation de l’application.

Je remercie également les auteurs des documentations et les communautés des outils utilisés. Leurs ressources m’ont permis d’approfondir mes connaissances en développement web, en bases de données et en modélisation, puis de les appliquer à MANUGOM Stock.

Je remercie enfin toutes les personnes qui ont contribué, directement ou indirectement, à l'aboutissement de ce travail. Les phases de vérification, les échanges autour des besoins et les essais de l'application ont permis de corriger plusieurs incohérences et de produire un prototype plus clair, documenté et démontrable.

Résumé

Le présent rapport décrit la conception et la réalisation de MANUGOM Stock, une application web destinée à centraliser un catalogue de pneumatiques, les commandes clients, les approvisionnements et les mouvements de stock. Le projet répond à un besoin de cohérence entre les références commerciales, les quantités disponibles et l'historique des opérations. Je l'ai développé comme projet personnel pour mettre en pratique la conception et la réalisation d'une application web complète.

La solution repose sur Django 6.1.1, une base SQLite unique, des templates HTML, Bootstrap et JavaScript. Elle distingue le visiteur, le client, l'employé et l'administrateur. Le visiteur consulte le catalogue et transmet une demande de devis. Le client gère son profil, son panier et ses commandes. L'employé traite les commandes et les stocks. L'administrateur dispose en plus de la gestion des utilisateurs et de l'administration Django.

Les opérations sensibles sont regroupées dans des services métier. Une commande reste en attente jusqu'à sa confirmation ; la disponibilité est alors vérifiée une seconde fois avant la création des sorties de stock. La réception d'un approvisionnement commandé produit les entrées correspondantes. Chaque mouvement conserve le stock avant et après l'opération. La version documentée a passé 34 tests automatisés et un contrôle d'intégrité SQLite. Ces résultats valident les scénarios couverts, sans constituer un essai de charge ni une preuve d'absence totale d'anomalies.

Mots-clés : Django, SQLite, gestion de stock, pneumatiques, commandes, approvisionnements, UML, contrôle d'accès, tests logiciels.

Abstract

This report presents the design and implementation of MANUGOM Stock, a web application that centralises a tyre catalogue, customer orders, procurement records and stock movements. The project addresses the need to keep commercial references, available quantities and operation history consistent. I developed it as a personal project to practise designing, implementing and testing a complete web application.

The solution uses Django 6.1.1, a single SQLite database, server-rendered HTML templates, Bootstrap and JavaScript. It distinguishes public visitors, customers, employees and administrators. Visitors browse the catalogue and submit quotation requests. Customers maintain a profile, cart and order history. Employees process orders and stock operations, while administrators also manage users and the technical Django administration area.

Sensitive operations are implemented in business services. An order remains pending until an employee confirms it; stock is checked again before quantities are removed. Receiving an ordered procurement record creates the corresponding stock entries. Each movement records quantities before and after the operation. The documented version passed 34 automated tests and an SQLite integrity check. These results validate the executed scenarios but do not constitute a load test or a proof that the system is defect-free.

Keywords: Django, SQLite, inventory management, tyres, orders, procurement, UML, access control, software testing.

Table des matières

Remerciements 2

Résumé 3

Abstract 4

Table des matières 5

Liste des figures 8

Liste des tableaux 9

Liste des abréviations 10

Introduction générale 1

Chapitre 1 - Contexte et démarche du projet 2

1.1 Présentation du projet personnel 2

1.2 Démarche technique 2

1.3 Présentation du contexte MANUGOM 2

1.4 Gestion de stock et apport de la recherche 3

Chapitre 2 - Présentation du projet 4

2.1 Problématique 4

2.2 Objectifs 4

2.3 Acteurs et responsabilités 4

2.4 Cahier des charges fonctionnel 5

2.5 Exigences non fonctionnelles 5

2.6 Règles de gestion 6

2.7 Organisation du travail 6

Chapitre 3 - Analyse et conception 7

3.1 Méthode de conception 7

3.2 Cas d'utilisation du visiteur et du client 8

3.3 Cas d'utilisation de la gestion 9

3.4 Architecture applicative 10

3.5 Classes du catalogue et des commandes 11
3.6 Classes des approvisionnements 12
3.7 Classes de traçabilité 13
3.8 Comptes et profil commercial 14
3.9 Modèle relationnel 14
3.10 Séquence de création d'une commande 16
3.11 Séquence de confirmation 17
3.12 Activité de traitement 18
3.13 Déploiement local 19
Chapitre 4 - Réalisation et développement 20
4.1 Environnement de développement 20
4.2 Organisation du code 20
4.3 Authentification et contrôle des rôles 21
4.4 Interfaces publiques 22
4.5 Espace client 25
4.6 Espace employé 27
4.7 Tableau de bord 29
4.8 Administration 31
4.9 Cohérence des opérations de stock 32
4.10 Difficultés rencontrées et corrections 33
Chapitre 5 - Résultats, tests et validation 34
5.1 Stratégie de test 34
5.2 Résultats observés 34
5.3 Validation par rapport aux objectifs 35
5.4 Limites actuelles 35
Conclusion générale et perspectives 36
Bibliographie 37

Annexes 38
Annexe A - Installation et lancement 38
Annexe B - Comptes de démonstration 38
Annexe C - Variables de configuration 38
Annexe D - Transitions principales 39

Liste des figures

Figure 1 : Cas d'utilisation du visiteur et du client 8
Figure 2 : Cas d'utilisation de l'employé et de l'administrateur 9
Figure 3 : Architecture Django MVT de MANUGOM Stock 10
Figure 4 : Classes du catalogue, du panier et des commandes 11
Figure 5 : Classes des fournisseurs et approvisionnements 12
Figure 6 : Classes de traçabilité du stock 13
Figure 7 : Classes des comptes, profils et clients 14
Figure 8 : Séquence de création depuis le panier 16
Figure 9 : Séquence de confirmation par un gestionnaire 17
Figure 10 : Activité du panier à la confirmation 18
Figure 11 : Déploiement de démonstration 19
Figure 12 : Page d'accueil publique 22
Figure 13 : Catalogue et filtres de recherche 23
Figure 14 : Fiche détaillée d'un pneu 24
Figure 15 : Historique des commandes du client 25
Figure 16 : Détail d'une commande client 26
Figure 17 : Suivi des commandes par l'employé 27
Figure 18 : Gestion du référentiel des pneus 28
Figure 19 : Indicateurs et ventes du tableau de bord 29
Figure 20 : Analyses complémentaires du stock 30
Figure 21 : Gestion des utilisateurs et des rôles 31
Figure 22 : Administration technique Django 32

Liste des tableaux

Tableau 1 : Abréviations utilisées 10
Tableau 2 : Acteurs du système 4
Tableau 3 : Fonctionnalités attendues 5
Tableau 4 : Exigences de qualité 5
Tableau 5 : Principales tables et clés 14
Tableau 6 : Environnement technique 20
Tableau 7 : Résultats des contrôles 34
Tableau 8 : Jeu de données présent dans SQLite 34
Tableau 9 : Validation des objectifs 35
Tableau 10 : Comptes fournis par seed_demo 38

Liste des abréviations

Tableau 1 : Abréviations utilisées

Abréviation	Signification
ACID	Atomicité, cohérence, isolation et durabilité
CRUD	Créer, consulter, modifier et supprimer
CSRF	Cross-Site Request Forgery
FK	Clé étrangère
HTTP	Hypertext Transfer Protocol
IIR	Ingénierie Informatique et Réseaux
MCD	Modèle conceptuel de données
MLD	Modèle logique de données
MVT	Model View Template
ORM	Object Relational Mapping
RBAC	Role-Based Access Control
UML	Unified Modeling Language

Introduction générale

La gestion d'un stock de pneumatiques ne se limite pas à afficher une quantité. Une référence associe une marque, un modèle, une dimension, des indices techniques, une saison, un prix et un état commercial. Une commande ajoute une autre contrainte : le système doit conserver ce que le client a commandé tout en tenant compte du stock qui peut évoluer entre la constitution du panier et le traitement par le magasin.

MANUGOM Stock a été conçu pour réunir ces informations dans une application web unique. Le catalogue est accessible au public, tandis que les opérations de commande et de gestion sont protégées par l'authentification et les rôles. Le projet cherche surtout à rendre les opérations explicables. Chaque entrée, sortie, retour ou ajustement produit une trace associée à un utilisateur et, lorsque c'est pertinent, à une commande ou à un approvisionnement.

La démarche a commencé par l'analyse des acteurs et des règles de gestion. Les modèles de données et les diagrammes UML ont ensuite servi à préciser les relations et les principaux scénarios. Le développement Django a été organisé par domaines fonctionnels, avec des services dédiés aux transitions sensibles. Enfin, les tests de modèles, de services, de vues et de permissions ont été exécutés sur une base temporaire.

Le rapport suit cette progression. Le premier chapitre présente le contexte et la démarche du projet. Le deuxième définit le projet, ses objectifs et son cahier des charges. Le troisième expose l'analyse, l'architecture et la conception UML. Le quatrième décrit la réalisation à l'aide de captures réelles de l'application. Le cinquième présente les tests, les résultats et les limites. La bibliographie réunit des articles scientifiques et les documentations techniques utilisées.

Chapitre 1 - Contexte et démarche du projet

1.1 Présentation du projet personnel

J'ai réalisé MANUGOM Stock pour mettre en pratique le développement d'une application web de gestion. Le projet part d'un besoin concret : réunir le catalogue de pneus, les commandes et les mouvements de stock dans un même outil. Ce rapport présente les choix de conception, les fonctionnalités réalisées et les résultats des vérifications.

Le travail relie l'analyse du besoin, la modélisation des données, le développement des interfaces et la validation des règles métier. La documentation permet de comprendre le fonctionnement de l'application et de la lancer dans un environnement local.

1.2 Démarche technique

La réalisation associe la conception logicielle à la configuration de l'environnement d'exécution. Django structure le traitement applicatif, SQLite conserve les données et HTTP relie le navigateur au serveur. Les mécanismes de session et de protection CSRF contrôlent les actions des utilisateurs.

La modélisation UML a servi à clarifier les responsabilités avant de décrire le code. Les cas d'utilisation montrent ce que chaque acteur peut demander au système. Les diagrammes de classes présentent les entités persistées et leurs relations. Les séquences et l'activité détaillent le moment où une commande devient confirmée et produit des mouvements de stock.

1.3 Présentation du contexte MANUGOM

MANUGOM SARL est présentée dans le projet comme une entreprise d'importation, de distribution et de vente de pneus et de jantes à Casablanca. Le catalogue de démonstration contient plusieurs marques, catégories et dimensions afin de représenter la diversité des références que doit traiter un magasin spécialisé. Les prix et stocks chargés par le script seed_demo sont des données de démonstration, clairement signalées dans l'interface.

Le point de départ est un besoin de centralisation. Sans un outil commun, le catalogue, la disponibilité, les commandes et les réceptions fournisseurs peuvent être suivis dans des supports différents. Cette dispersion complique la recherche d'une référence, le contrôle d'une quantité et l'explication d'un écart de stock. L'application rassemble ces opérations tout en conservant un historique exploitable.

1.4 Gestion de stock et apport de la recherche

La gestion des stocks est un domaine ancien de la recherche opérationnelle. Harris [6] a formalisé le compromis entre la taille des lots et les coûts associés au stockage et au lancement d'une opération. MANUGOM Stock n'implémente pas de calcul de quantité économique ni de prévision de demande. Il fournit les données élémentaires nécessaires à une future étude : quantités, seuils, entrées, sorties, dates et références concernées.

Dans la version actuelle, l'aide au réapprovisionnement repose sur un seuil minimum. Le tableau de bord signale les stocks faibles et les ruptures, mais la décision d'achat appartient encore à l'employé ou à l'administrateur. Cette limite est cohérente avec le périmètre de ce projet et évite de présenter une règle simple comme un modèle d'optimisation scientifique.

Chapitre 2 - Présentation du projet

2.1 Problématique

Comment construire une application qui permette de consulter précisément les pneumatiques disponibles, de transformer un panier en commande et de mettre à jour le stock sans perdre l'historique des opérations ? Cette question implique trois exigences liées. Les références doivent être assez structurées pour être recherchées. Les rôles doivent limiter les actions. Les transitions de commande et d'approvisionnement doivent rester cohérentes avec les mouvements de stock.

2.2 Objectifs

L'objectif général est de proposer un prototype web démontrable qui centralise le catalogue, les clients, les commandes, les fournisseurs, les approvisionnements et les mouvements. Le projet vise également à rendre la logique vérifiable par des tests et à fournir une documentation de lancement utilisable sur un poste Windows.

- Permettre une recherche par référence, marque, catégorie, dimension, saison, prix et disponibilité.
- Donner au client un panier persistant et un historique limité à ses propres commandes.
- Fournir aux gestionnaires les écrans nécessaires au traitement des commandes et des stocks.
- Tracer chaque variation avec le stock avant et après l'opération.
- Conserver une base SQLite unique et des commandes d'installation simples.

2.3 Acteurs et responsabilités

Tableau 2 : Acteurs du système

Acteur	Responsabilités principales	Restrictions
Visiteur	Consulter le site, rechercher un pneu, demander un devis et envoyer un message.	Aucun panier ni commande.
Client	Gérer son profil, son panier et ses commandes.	Accès à ses propres commandes seulement.
Employé	Gérer les références, commandes, approvisionnements, mouvements et alertes.	Aucune gestion des comptes administrateurs.

Acteur	Responsabilités principales	Restrictions
Administrateur	Superviser l'activité, gérer les utilisateurs et accéder à l'administration Django.	Actions critiques toujours contrôlées côté serveur.

2.4 Cahier des charges fonctionnel

Tableau 3 : Fonctionnalités attendues

Domaine	Fonctions
Catalogue	Recherche, filtres, tri, pagination, détail et disponibilité.
Compte client	Inscription, connexion, profil et isolation des données.
Panier	Ajout, modification, retrait, contrôle des quantités et calcul du total.
Commandes	Création en attente, confirmation, préparation, expédition, livraison et annulation autorisée.
Stock	Entrées, sorties, retours, ajustements, seuils et historique.
Approvisionnements	Fournisseur, lignes, statut commandé et réception unique.
Pilotage	Indicateurs, graphiques, devis, contacts et gestion des utilisateurs.

2.5 Exigences non fonctionnelles

Tableau 4 : Exigences de qualité

Exigence	Réponse du projet	Limite
Sécurité	Sessions Django, CSRF, validation serveur, rôles et filtrage par propriétaire.	Configuration locale de démonstration.
Intégrité	Transactions courtes, contraintes d'unicité, stock non négatif et traces métier.	Pas d'essai multiconnexion.
Utilisabilité	Interface responsive, navigation séparée par rôle et messages de résultat.	Pas d'étude utilisateur formelle.
Maintenabilité	Découpage en applications et services métier testés.	Prototype local sans CI distante.
Portabilité	SQLite, requirements.txt, migrations et seed_demo.	Serveur de développement local.

2.6 Règles de gestion

- Le panier ne réserve pas la quantité disponible.
- La création produit une commande EN_ATTENTE et désactive le panier utilisé.
- La confirmation revérifie le stock avant toute sortie.
- Une annulation après confirmation restaure les quantités une seule fois si la préparation n'a pas commencé.
- Seul un approvisionnement au statut COMMANDE et non encore reçu peut être réceptionné.
- Une ligne de commande conserve la référence, la dimension et le prix observés lors de la vente.

2.7 Organisation du travail

Le développement a suivi des itérations courtes : initialisation de la base et des rôles, création du catalogue, mise en place du panier et des commandes, ajout des approvisionnements, construction du tableau de bord, puis tests et documentation. Les corrections ont été intégrées au même projet afin que le code, les captures et le rapport décrivent une seule version.

Chapitre 3 - Analyse et conception

3.1 Méthode de conception

L'analyse part des opérations observables puis descend vers les données et les services. Le découpage évite un diagramme global illisible. Deux vues de cas d'utilisation séparent le parcours public et client de l'espace de gestion. Les classes sont réparties par domaine. Les séquences distinguent la création d'une commande de sa confirmation, car le stock n'est retiré qu'au second moment.

Parnas [2] montre que la valeur d'une décomposition dépend des responsabilités confiées à chaque module. Ce principe éclaire ici la séparation entre comptes, inventaire, clients, commandes, fournisseurs et tableau de bord. Les services ServiceCommande, ServiceStock et ServiceApprovisionnement regroupent les changements d'état sensibles, tandis que les vues orchestrent les requêtes HTTP et rendent les templates.

3.2 Cas d'utilisation du visiteur et du client

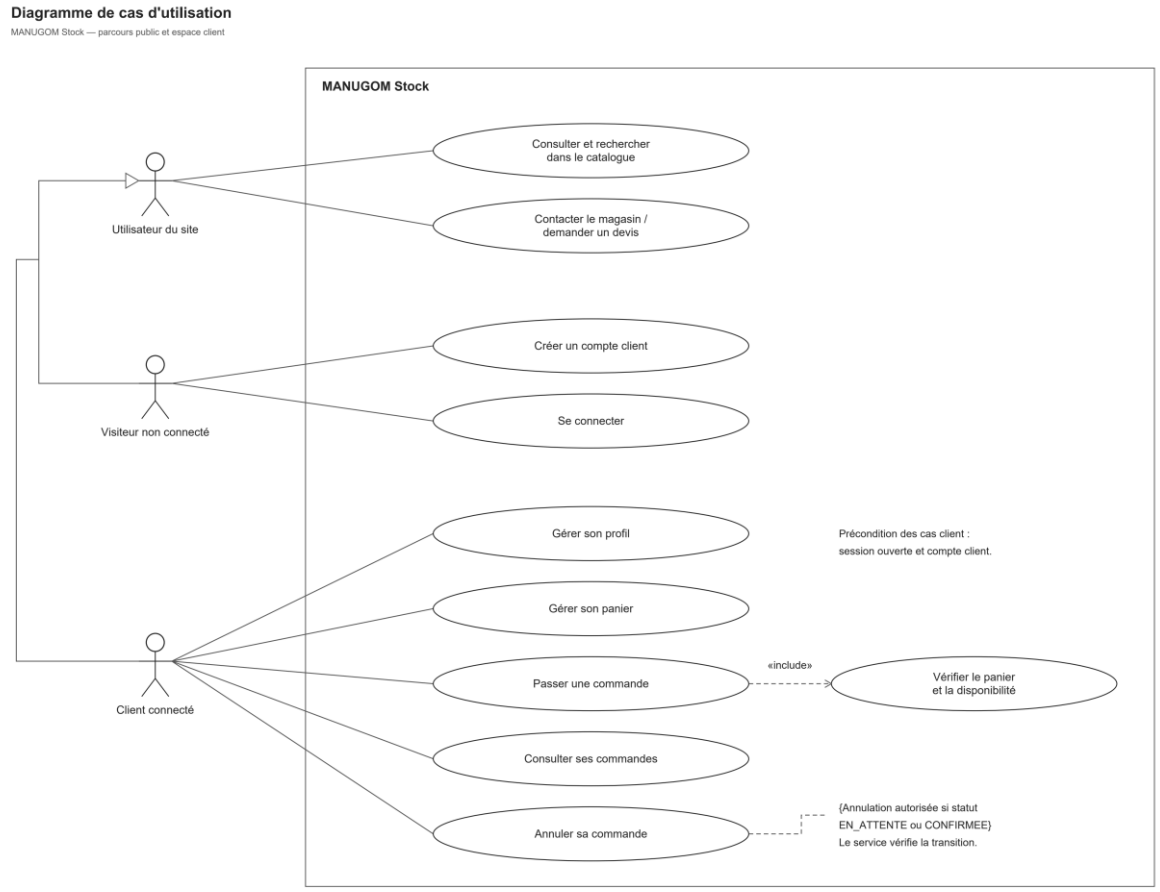


***Figure 1* : Cas d'utilisation du visiteur et du client**

Interprétation de la figure. Utilisateur du site regroupe les fonctions publiques : rechercher dans le catalogue et contacter le magasin ou demander un devis. Visiteur non connecté et Client connecté spécialisent cet acteur et héritent de ces fonctions. Les triangles blancs pointent vers l'acteur général. La création de compte et la connexion appartiennent au visiteur ; le client dispose du profil, du panier et de ses commandes. La relation include impose la vérification du panier et de la disponibilité lors du passage d'une commande. L'annulation reste soumise au statut de la commande.

3.3 Cas d'utilisation de la gestion

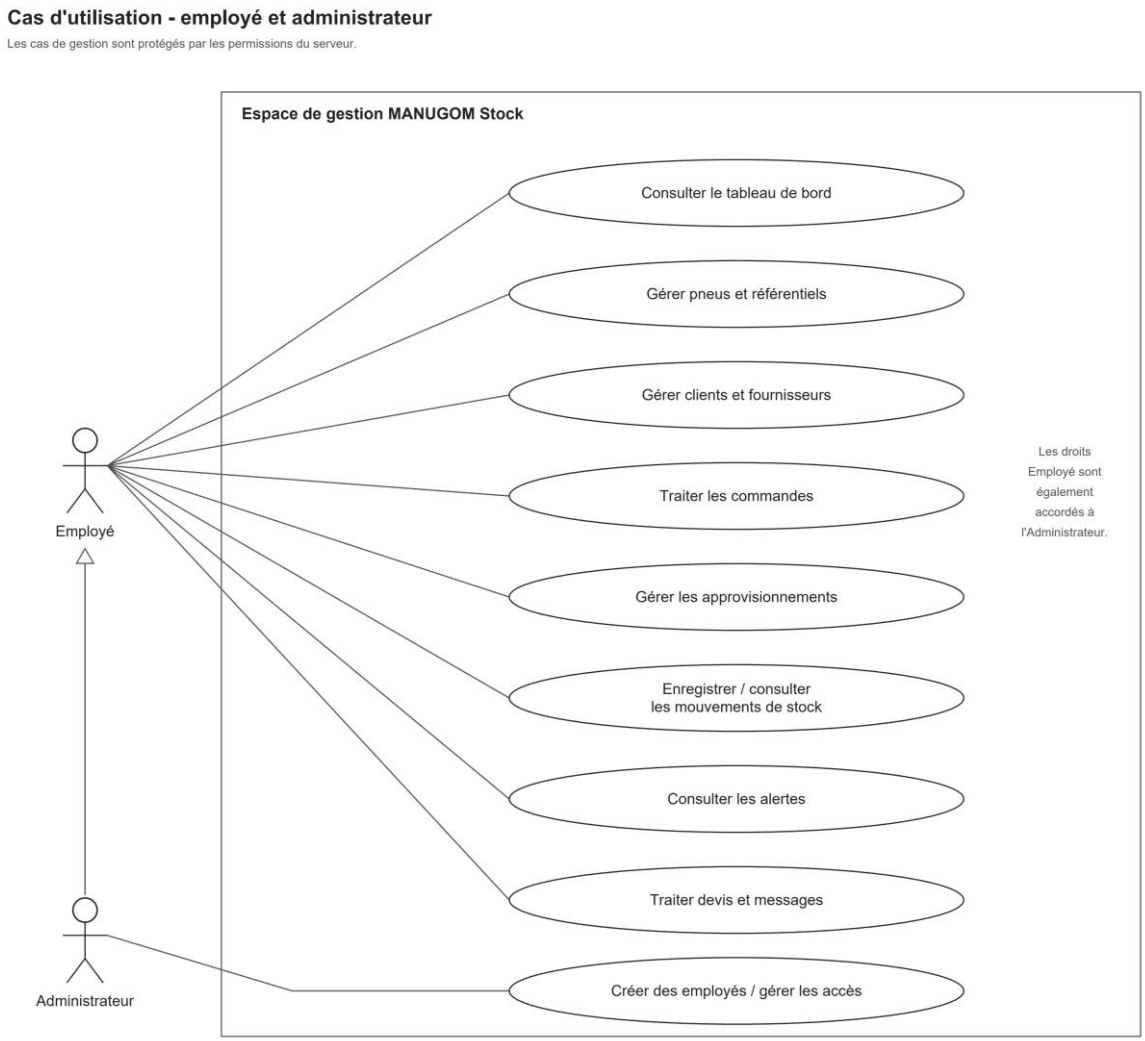


Figure 2 : Cas d'utilisation de l'employé et de l'administrateur

Interprétation de la figure. L'employé couvre le fonctionnement quotidien du magasin. L'administrateur hérite de ces possibilités et ajoute la gestion des utilisateurs et l'accès technique à Django Admin. Les associations sont individuelles et ne traversent pas les cas d'utilisation, ce qui permet de lire la figure sans ambiguïté.

3.4 Architecture applicative

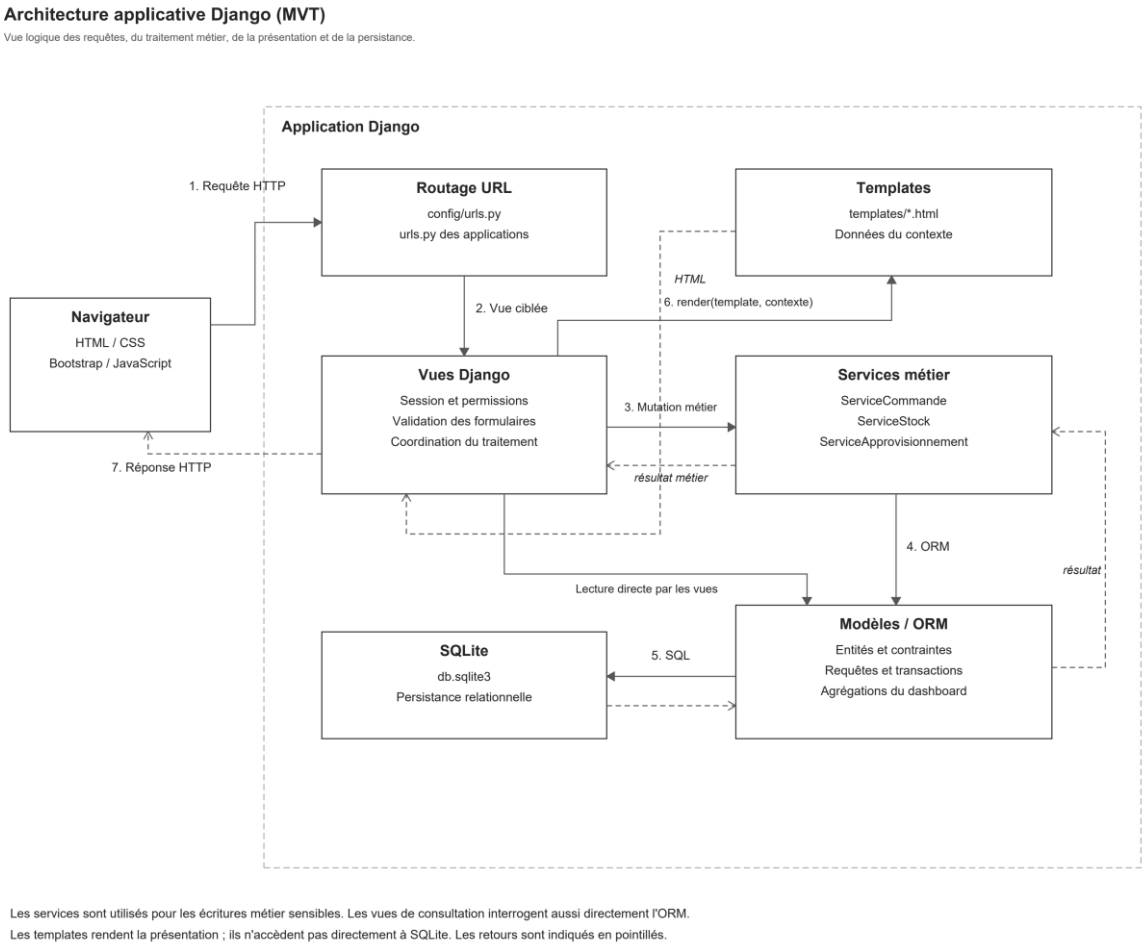


Figure 3 : Architecture Django MVT de MANUGOM Stock

Interprétation de la figure. La requête HTTP est dirigée par le routage vers une vue. La vue contrôle la session, les permissions et les formulaires. Elle peut appeler un service pour une mutation sensible ou interroger l'ORM pour une lecture. Le template reçoit un contexte et produit la réponse HTML. Les templates et le navigateur n'accèdent jamais directement à SQLite.

Le diagramme corrige une simplification fréquente : toutes les lectures ne passent pas par un service. Les pages de catalogue et le tableau de bord utilisent aussi l'ORM depuis les vues. Les services forment une couche volontaire pour les règles critiques, mais Django demeure une application MVT rendue côté serveur.

3.5 Classes du catalogue et des commandes

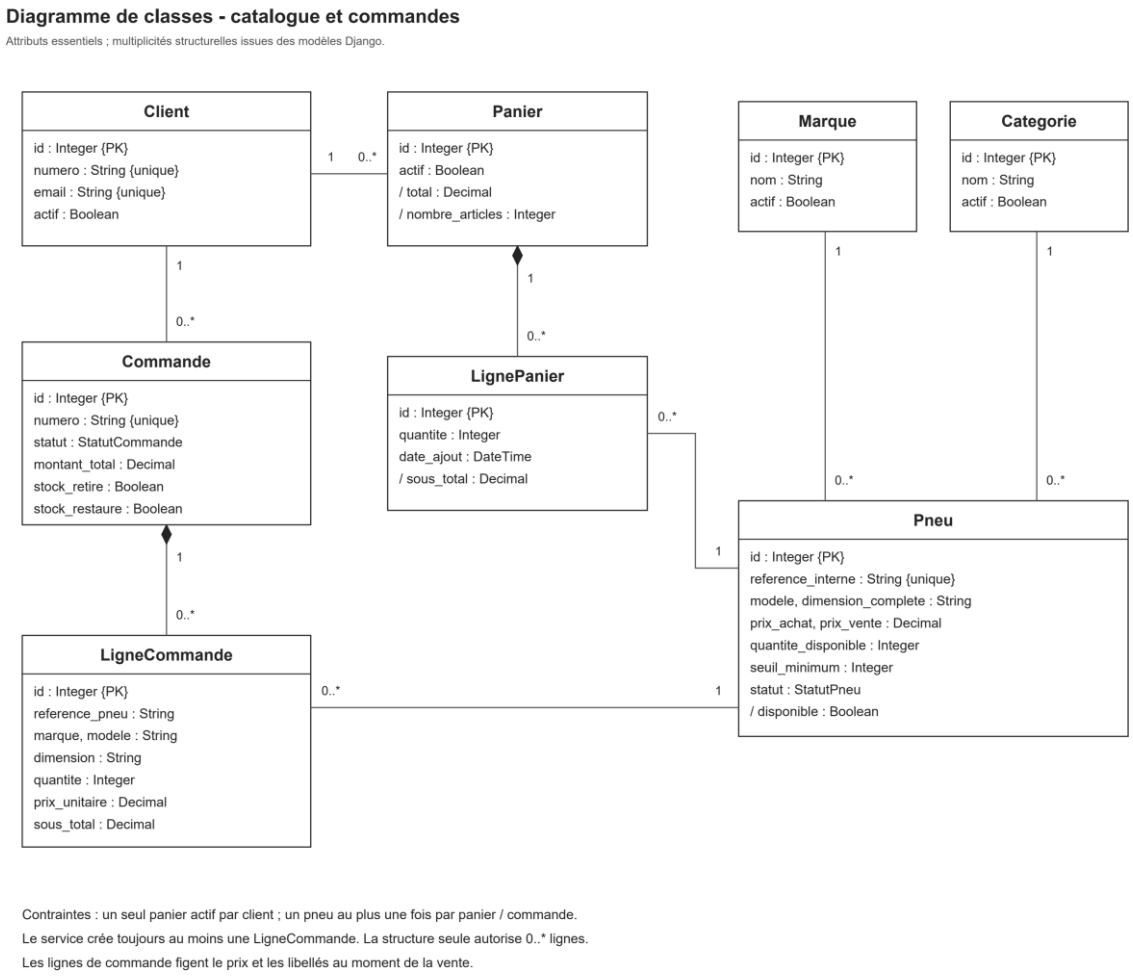


Figure 4 : Classes du catalogue, du panier et des commandes

Interprétation de la figure. Client possède ses paniers et commandes. Les compositions indiquent que les lignes dépendent de leur panier ou commande. Pneu est relié à une marque et une catégorie. LigneCommande contient des instantanés de libellé et de prix afin qu'une modification future du catalogue ne réécrive pas l'historique.

3.6 Classes des approvisionnements

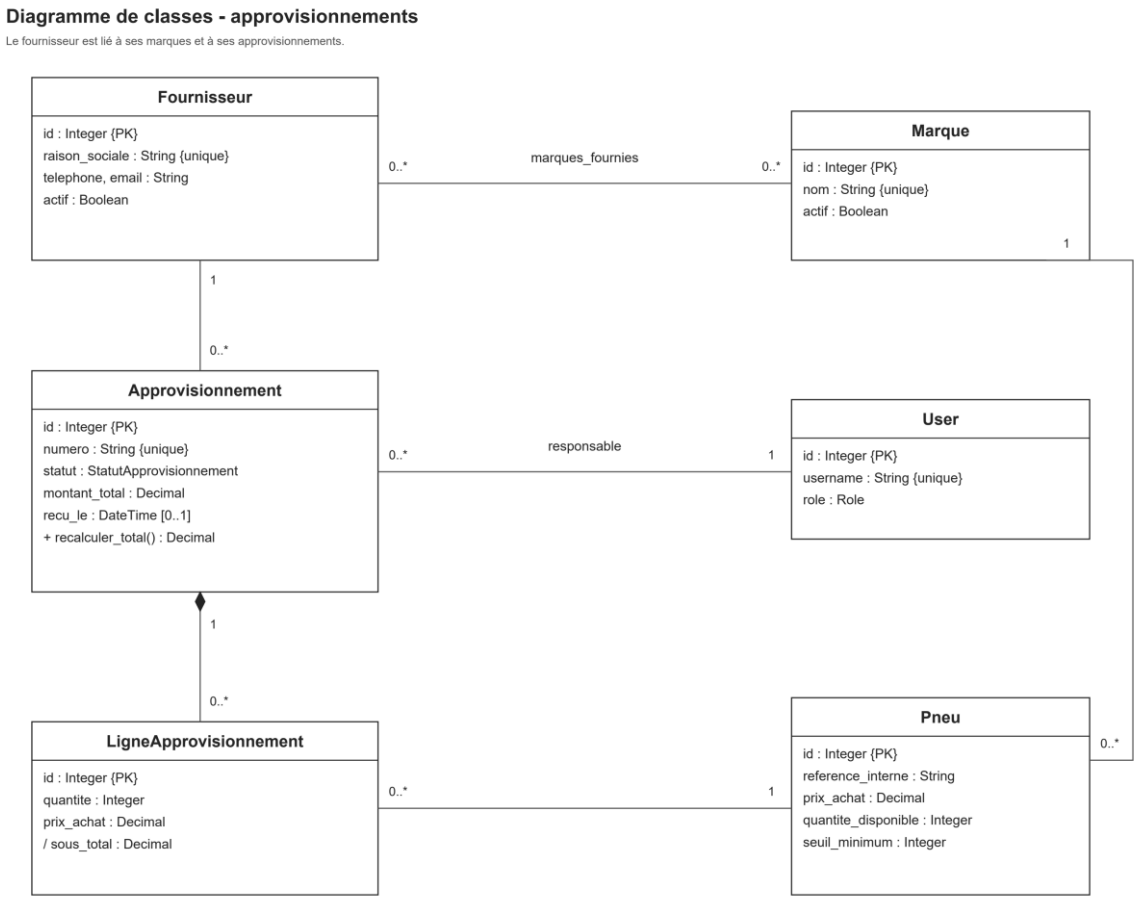


Figure 5 : Classes des fournisseurs et approvisionnements

Interprétation de la figure. Un fournisseur peut être lié à plusieurs marques et approvisionnements. Chaque approvisionnement contient ses lignes ; chaque ligne désigne un pneu et un prix d'achat. Le responsable est un utilisateur. La réception est réservée au statut COMMANDE et ne peut être répétée.

3.7 Classes de traçabilité

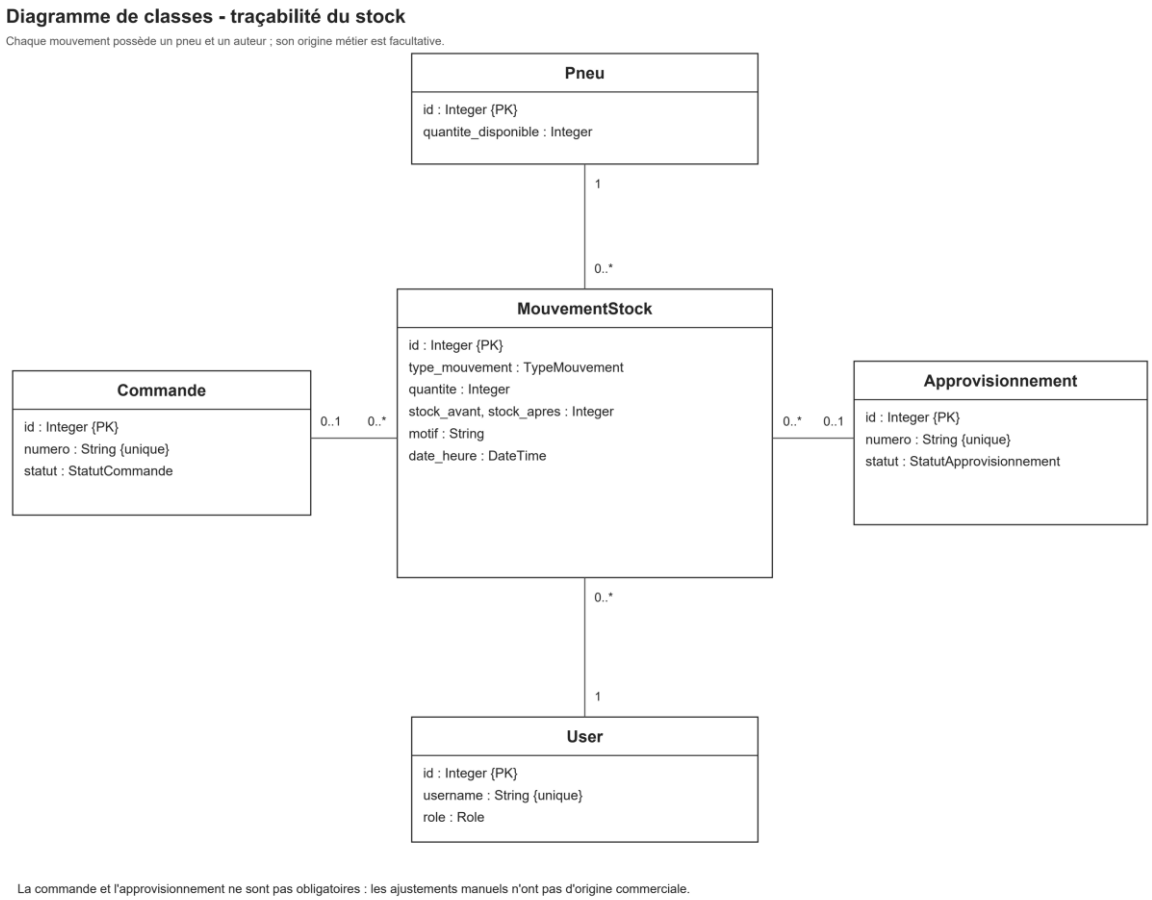


Figure 6 : Classes de traçabilité du stock

Interprétation de la figure. Tout mouvement concerne un pneu et un auteur. La commande et l'approvisionnement sont facultatifs afin d'autoriser les ajustements manuels. Les champs stock_avant et stock_apres fournissent une trace compréhensible sans recalculer tout l'historique.

3.8 Comptes et profil commercial

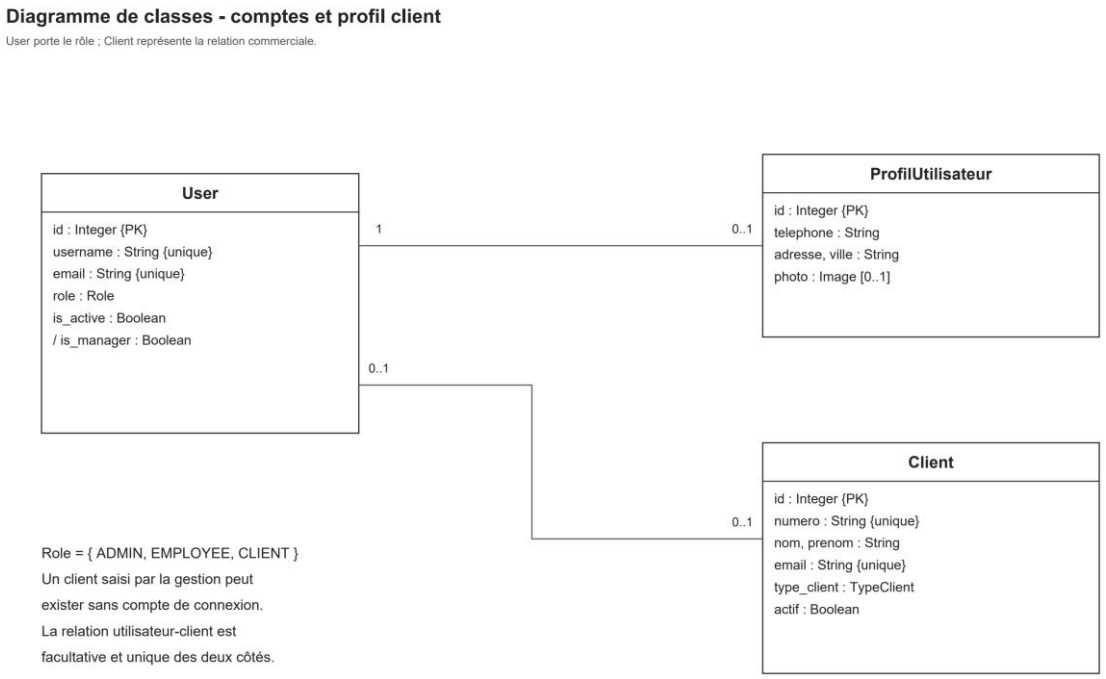


Figure 7 : Classes des comptes, profils et clients

Interprétation de la figure. User porte l'identité de connexion et le rôle. ProfilUtilisateur conserve les coordonnées associées au compte. Client représente la fiche commerciale ; elle peut exister sans compte lorsqu'elle est créée par un gestionnaire. La relation facultative évite de forcer une authentification pour toute fiche client.

3.9 Modèle relationnel

Le modèle relationnel de Codd [1] fournit le cadre conceptuel utilisé pour séparer les entités et exprimer leurs dépendances au moyen de clés. Le schéma de MANUGOM Stock conserve toutefois une redondance intentionnelle dans LigneCommande : la référence, le modèle, la dimension et le prix sont copiés au moment de la commande pour préserver l'historique.

Tableau 5 : Principales tables et clés

Table	Clé primaire	Clés étrangères principales	Contraintes
inventory_pneu	id	marque_id, categorie_id	reference unique, stock >= 0

Table	Clé primaire	Clés étrangères principales	Contraintes
orders_panier	id	client_id	un panier actif par client
orders_lignepanier	id	panier_id, pneu_id	paire panier et pneu unique
orders_commande	id	client_id	numero unique
orders_lignecommande	id	commande_id, pneu_id	instantanés de vente
inventory_mouvementstock	id	pneu, user, commande ou approvisionnement	quantité contrôlée
suppliers_approvisionnement	id	fournisseur, responsable	numero unique

3.10 Séquence de création d'une commande

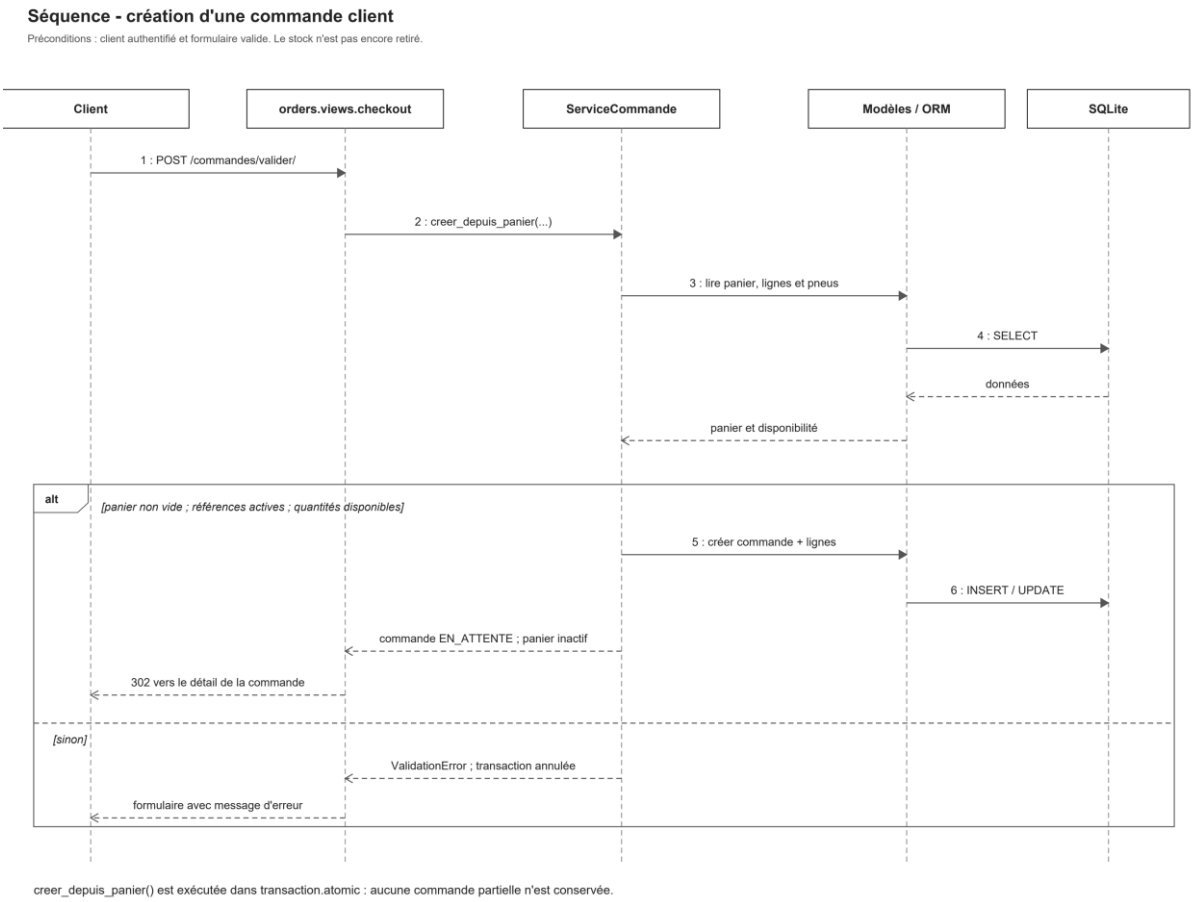


Figure 8 : Séquence de création depuis le panier

Interprétation de la figure. Le client envoie un formulaire POST. La vue délègue au service, qui relit les lignes et crée la commande et ses lignes dans une transaction. Le panier devient inactif. Aucun mouvement de sortie n'est encore produit ; l'échec d'une condition annule la transaction et renvoie un message.

3.11 Séquence de confirmation

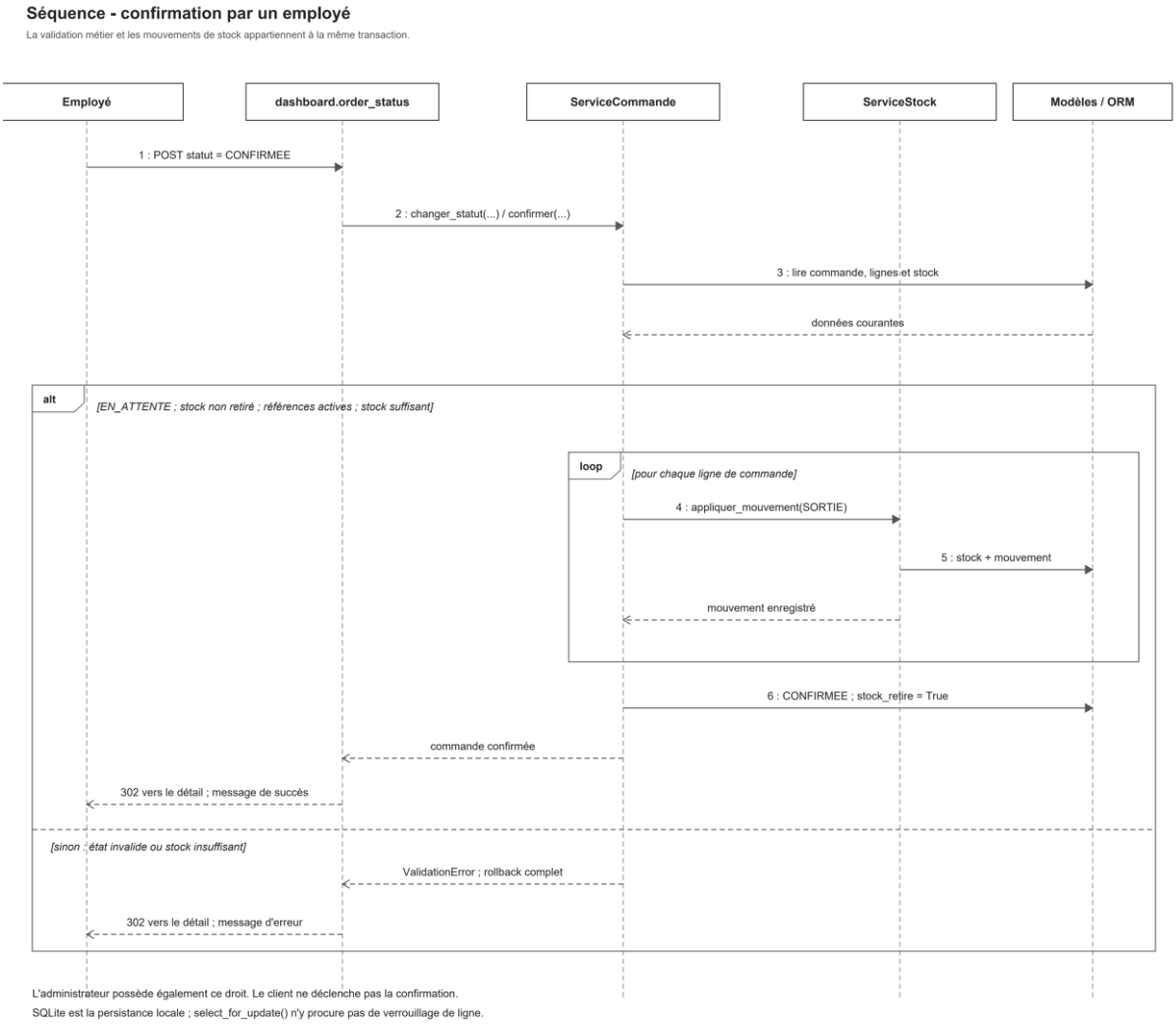


Figure 9 : Séquence de confirmation par un gestionnaire

Interprétation de la figure. L'employé ou l'administrateur demande la confirmation. Le service vérifie l'état et le stock, applique une sortie pour chaque ligne et marque la commande confirmée. Une exception provoque le rollback du groupe. Sous SQLite, select_for_update n'ajoute pas de verrou de ligne ; les tests ne simulent donc pas une forte concurrence.

Härder et Reuter [3] décrivent les propriétés ACID des transactions. Dans ce projet, transaction.atomic regroupe les écritures qui doivent réussir ensemble. Cette utilisation protège contre une commande partiellement confirmée à la suite d'une exception. Elle ne constitue pas, à elle seule, une validation du comportement sous charge concurrente.

3.12 Activité de traitement

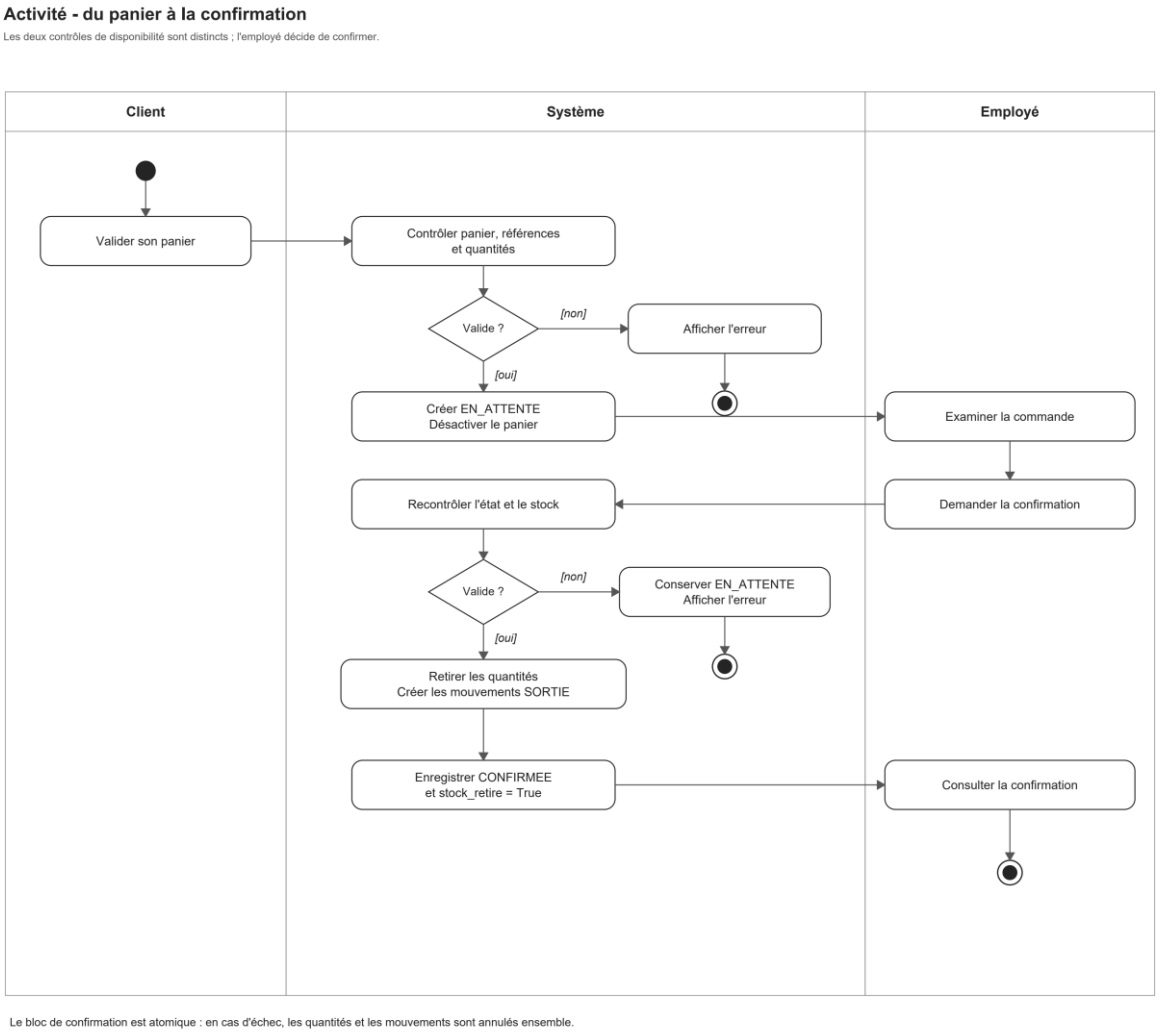


Figure 10 : Activité du panier à la confirmation

Interprétation de la figure. Le premier contrôle appartient au parcours client et crée une commande en attente. Le second appartient au gestionnaire et précède le retrait du stock. Les branches d'erreur se terminent sans modification partielle. Cette séparation explique pourquoi l'affichage du panier ne réserve pas une quantité.

3.13 Déploiement local

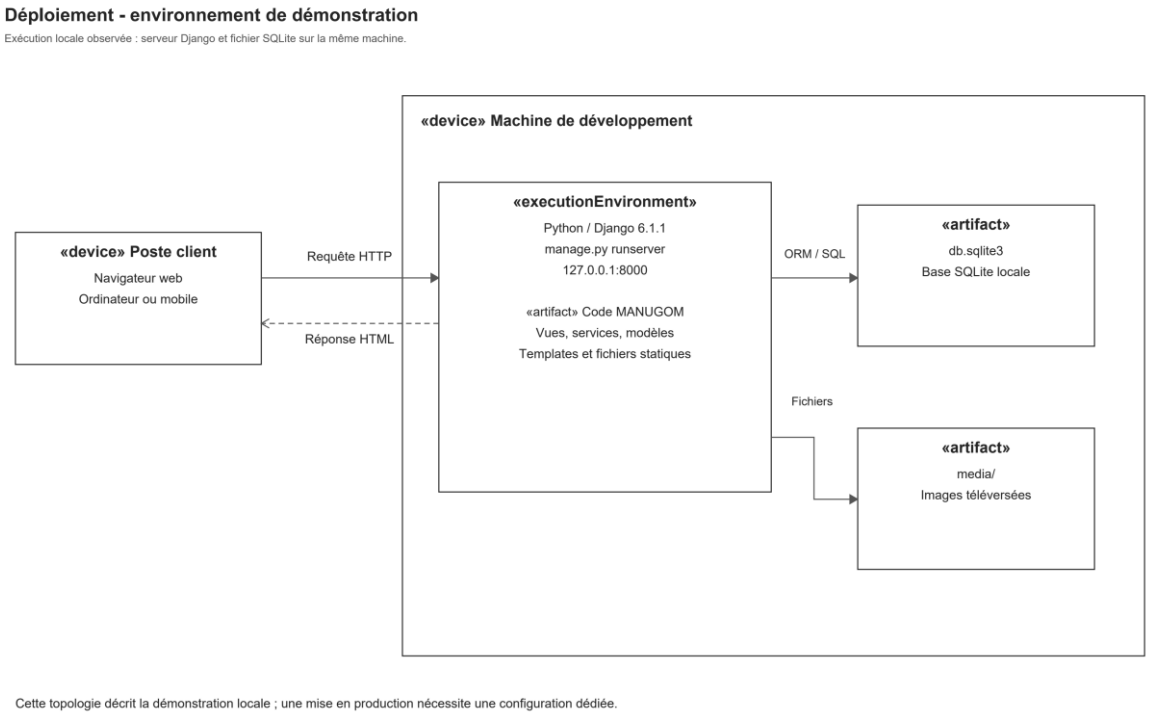


Figure 11 : Déploiement de démonstration

Interprétation de la figure. Le navigateur communique en HTTP avec le serveur Django local. Le code, SQLite et le dossier media résident sur la machine de développement. Cette topologie correspond à la démonstration ; une production demanderait un serveur applicatif, HTTPS, des secrets adaptés, un stockage sauvegardé et une base choisie selon la concurrence attendue.

Chapitre 4 - Réalisation et développement

4.1 Environnement de développement

Tableau 6 : Environnement technique

Élément	Version ou rôle
Python	3.14.6 dans l'environnement local contrôlé
Django	6.1.1
Pillow	12.3.0
SQLite	3.50.4 observé avec Python
Bootstrap	Mise en page responsive
Chart.js	Graphiques du tableau de bord
ReportLab et python-docx	Documentation et diagrammes

Les commandes principales restent volontairement simples. Elles créent l'environnement, installent les dépendances, préparent la base, chargent le jeu de démonstration et démarrent le serveur local.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py seed_demo
python manage.py runserver
```

4.2 Organisation du code

Le projet contient huit applications Django. accounts gère le compte personnalisé et les rôles. inventory contient les pneus, marques, catégories, mouvements et ServiceStock. customers représente les fiches clients. orders regroupe le panier, la commande et ServiceCommande. suppliers couvre les fournisseurs, les lignes d'approvisionnement et ServiceApprovisionnement. dashboard fournit l'espace de gestion. quotes enregistre les devis et contacts. core réunit les pages institutionnelles et la commande seed_demo.

Les URL sont regroupées par espace de noms. Les templates réutilisent une base commune et des composants de présentation. Les fichiers statiques sont centralisés dans static, tandis que

les images téléversées se trouvent dans media. Ce découpage limite le couplage entre domaines et rend les tests plus ciblés.

4.3 Authentification et contrôle des rôles

Le modèle User étend AbstractUser avec trois valeurs : ADMIN, EMPLOYEE et CLIENT. Les décorateurs manager_required et admin_required protègent les vues de gestion. Les détails de commande ajoutent un contrôle de propriétaire pour qu'un client ne puisse consulter qu'une commande reliée à sa propre fiche.

Sandhu, Ferraiolo et Kuhn [4] associent les autorisations aux rôles organisationnels dans le modèle RBAC. MANUGOM Stock applique une version simplifiée de cette idée. Il ne fournit pas un moteur RBAC configurable, une hiérarchie dynamique ni des contraintes formelles de séparation des tâches. Le visiteur est simplement un utilisateur anonyme.

4.4 Interfaces publiques

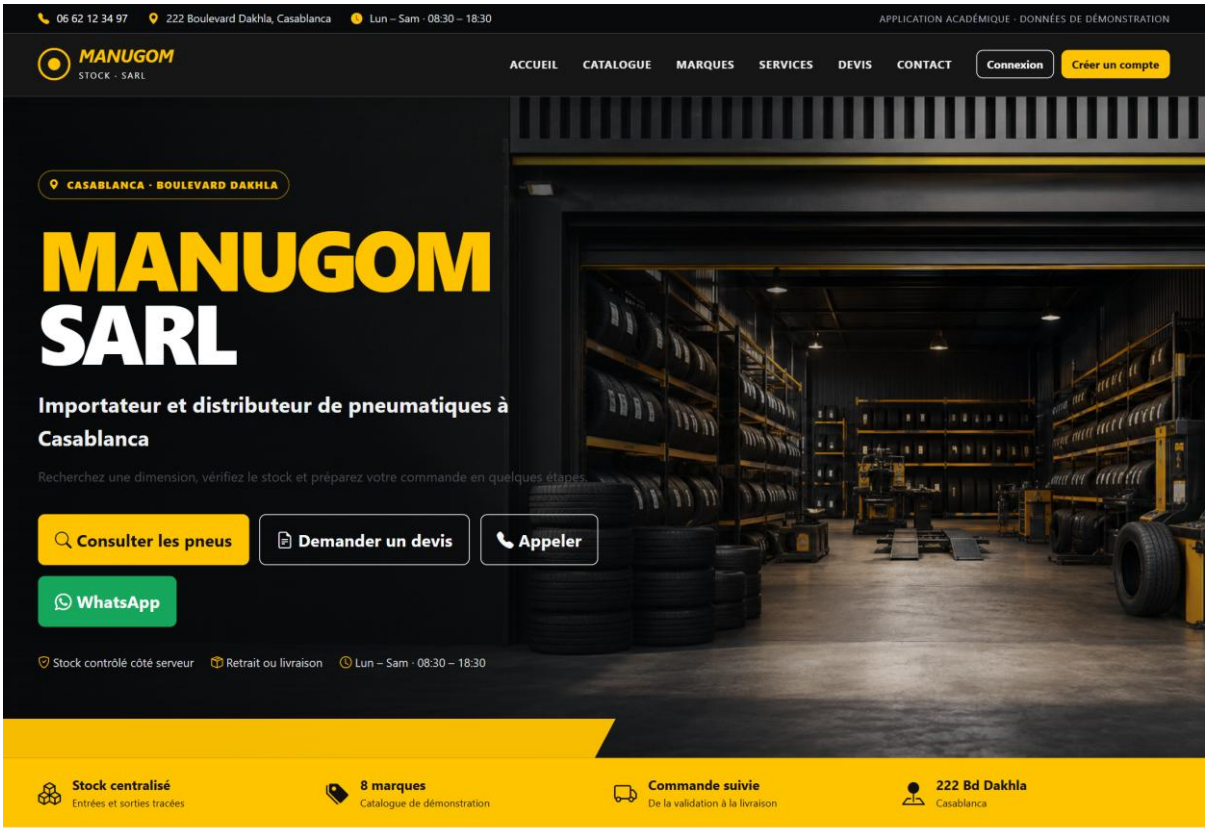


Figure 12 : Page d'accueil publique

Interprétation de la figure. La page présente l'activité, les coordonnées et quatre actions directes : catalogue, devis, appel et WhatsApp. Les liens téléphoniques utilisent tel: et wa.me. Ils ouvrent l'application correspondante sans appeler une API depuis Django.

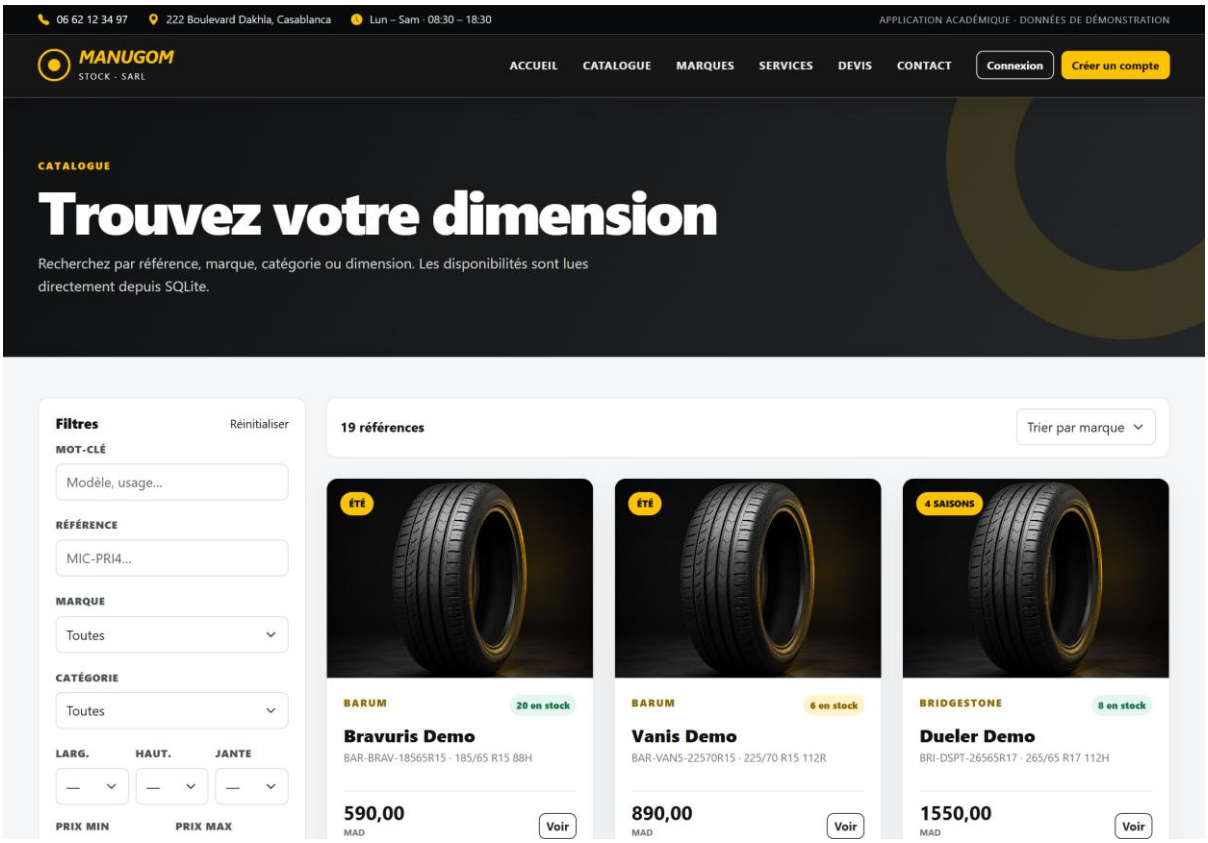


Figure 13 : Catalogue et filtres de recherche

Interprétation de la figure. Le visiteur recherche une référence sans connexion. Les filtres portent sur les dimensions, la marque, la catégorie, la saison, le prix et la disponibilité. Les quantités proviennent de SQLite et les cartes affichent explicitement des données de démonstration.

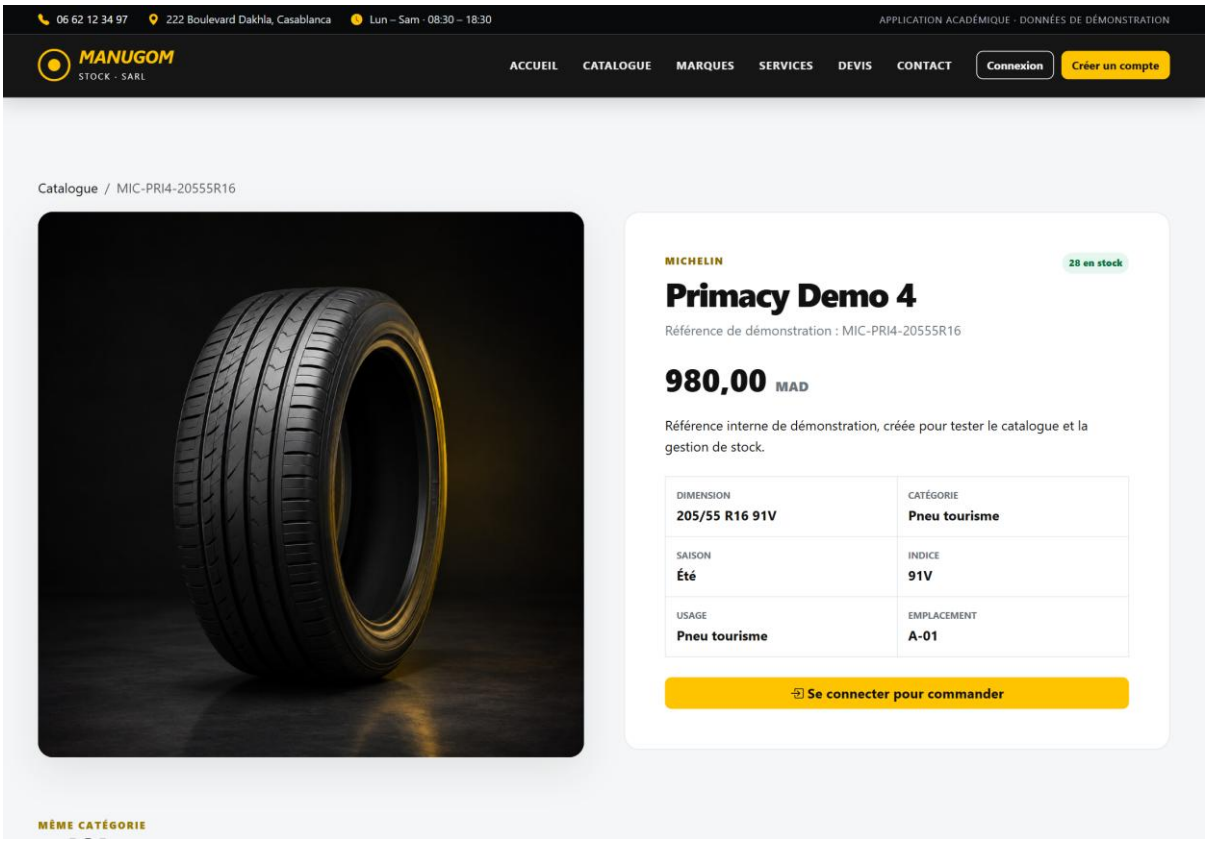


Figure 14 : Fiche détaillée d'un pneu

Interprétation de la figure. La fiche réunit les caractéristiques utiles avant une commande. Lorsqu'un visiteur souhaite ajouter l'article, le parcours lui propose de se connecter puis le ramène vers une page GET valide. Cette correction évite la réponse HTTP 405 observée lors d'une redirection vers une URL POST.

4.5 Espace client

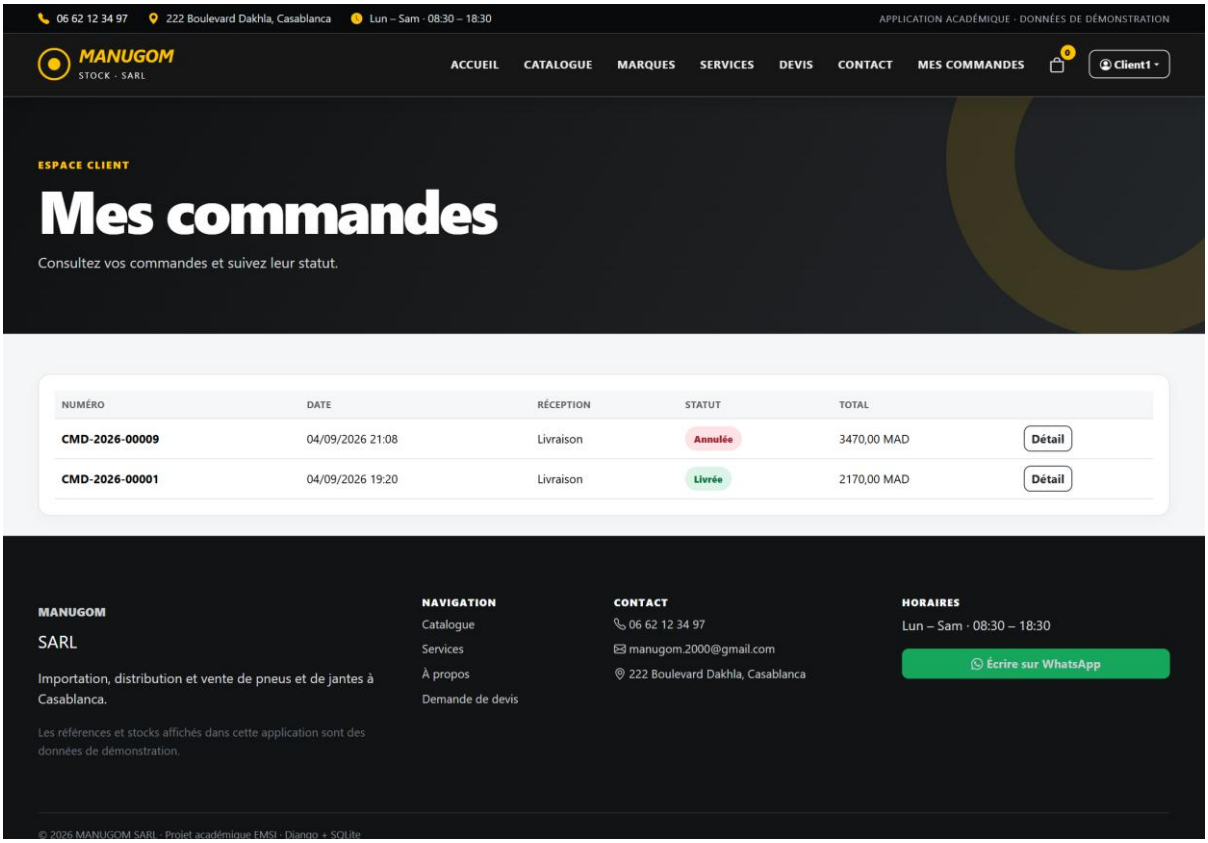


Figure 15 : Historique des commandes du client

Interprétation de la figure. Le compte client1 ne voit que ses propres commandes. Le numéro, la date, le mode de réception, le statut et le total permettent de suivre chaque demande sans exposer celles des autres clients.

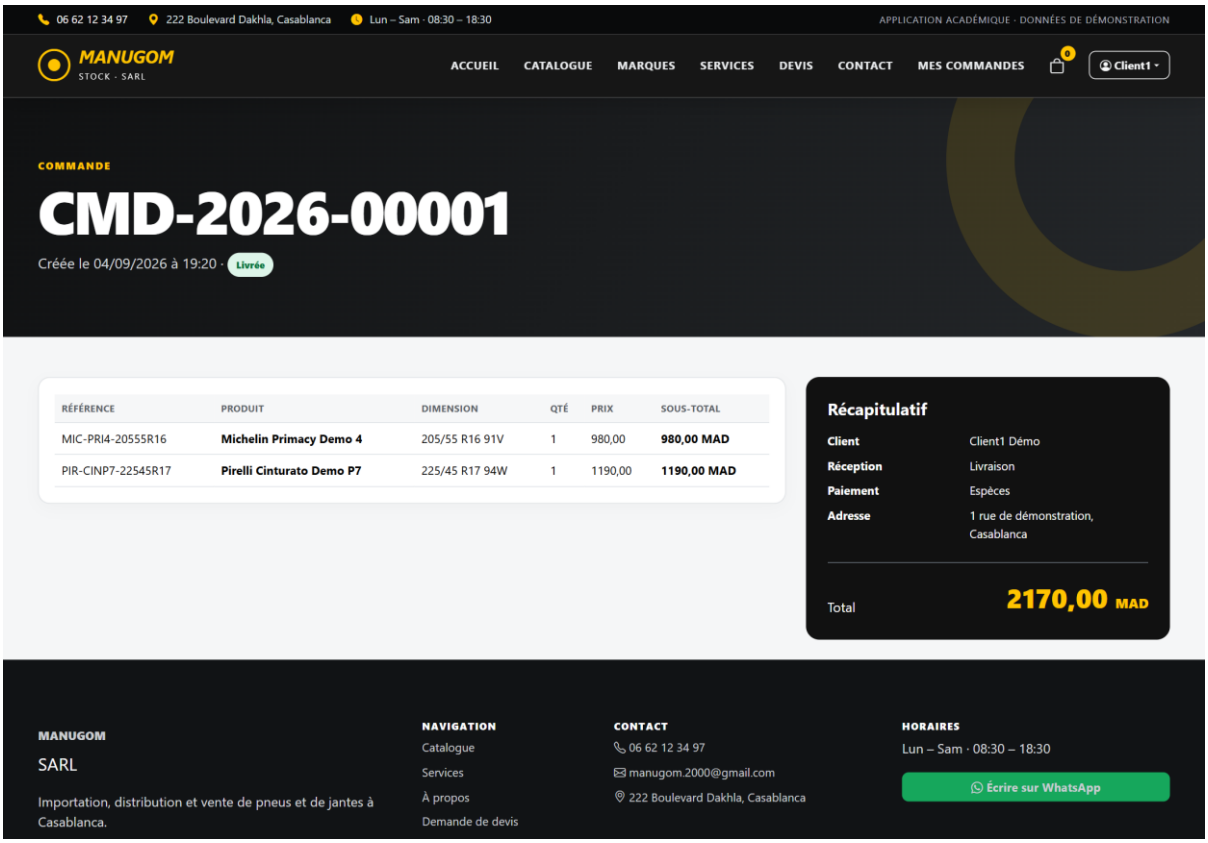


Figure 16 : Détail d'une commande client

Interprétation de la figure. Les lignes affichées reprennent les instantanés enregistrés lors de la création. Cette présentation reste stable même si un pneu change ensuite de prix ou de libellé dans le catalogue.

4.6 Espace employé

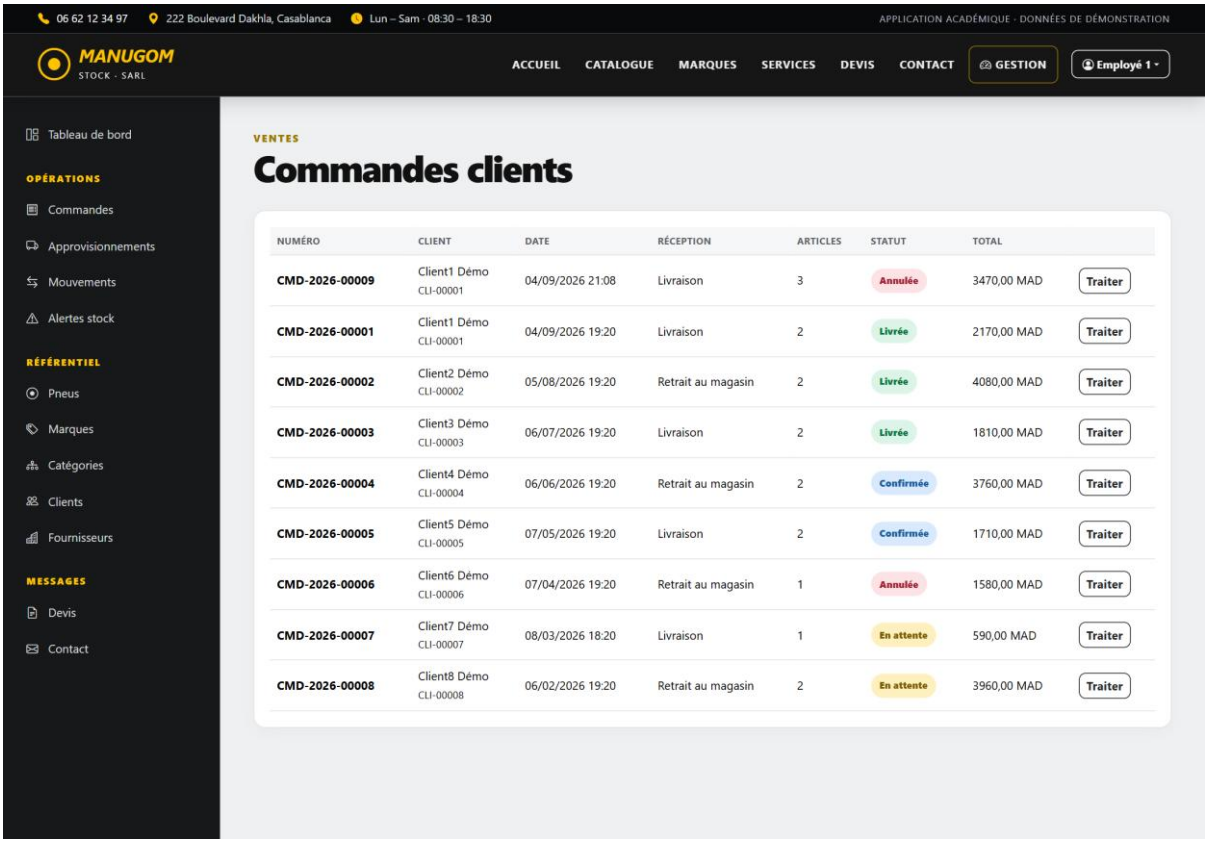


Figure 17 : Suivi des commandes par l'employé

Interprétation de la figure. La liste de gestion regroupe les commandes de tous les clients avec leur état et une action de traitement. Les transitions proposées par la fiche dépendent du statut courant et sont revérifiées côté serveur.

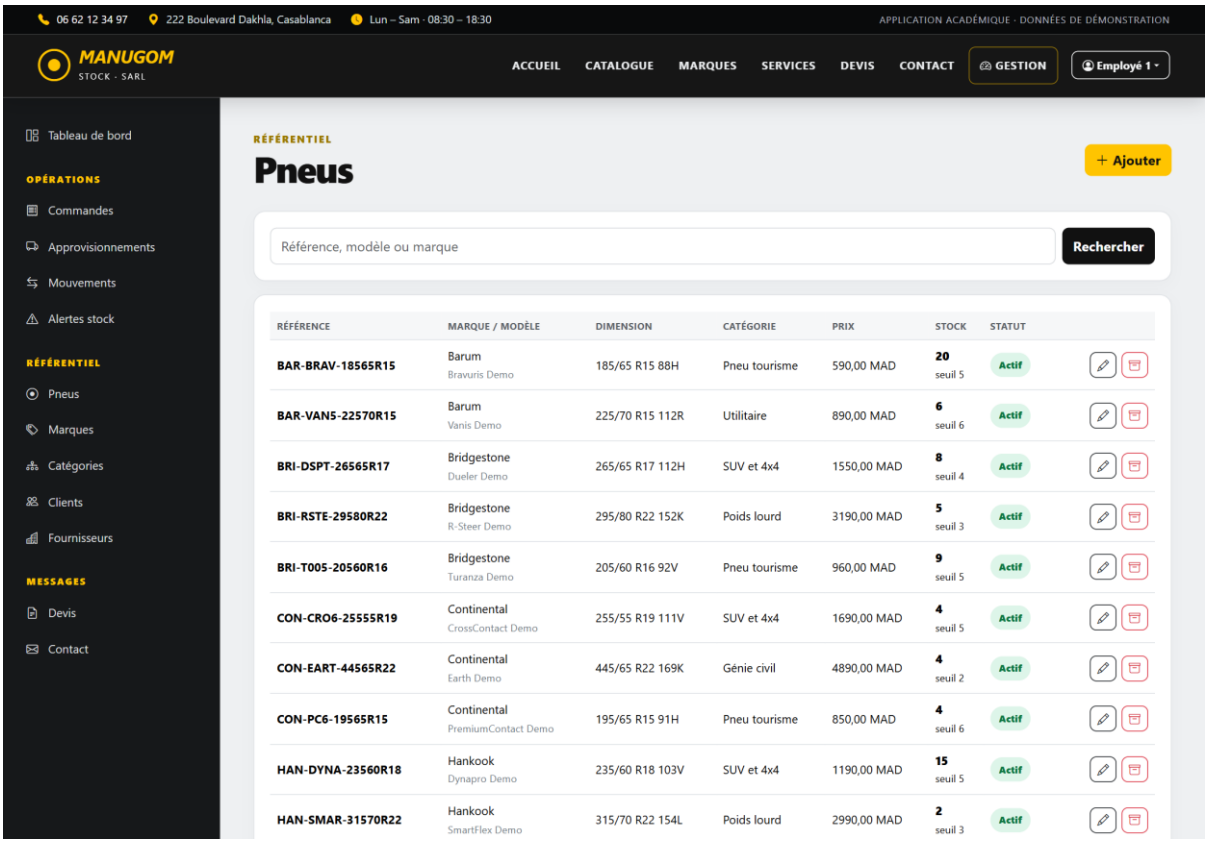


Figure 18 : Gestion du référentiel des pneus

Interprétation de la figure. L'employé consulte les stocks, les prix et les références depuis une vue compacte. Les formulaires contrôlent la dimension et les montants. L'archivage est privilégié lorsqu'une référence possède déjà un historique.

4.7 Tableau de bord

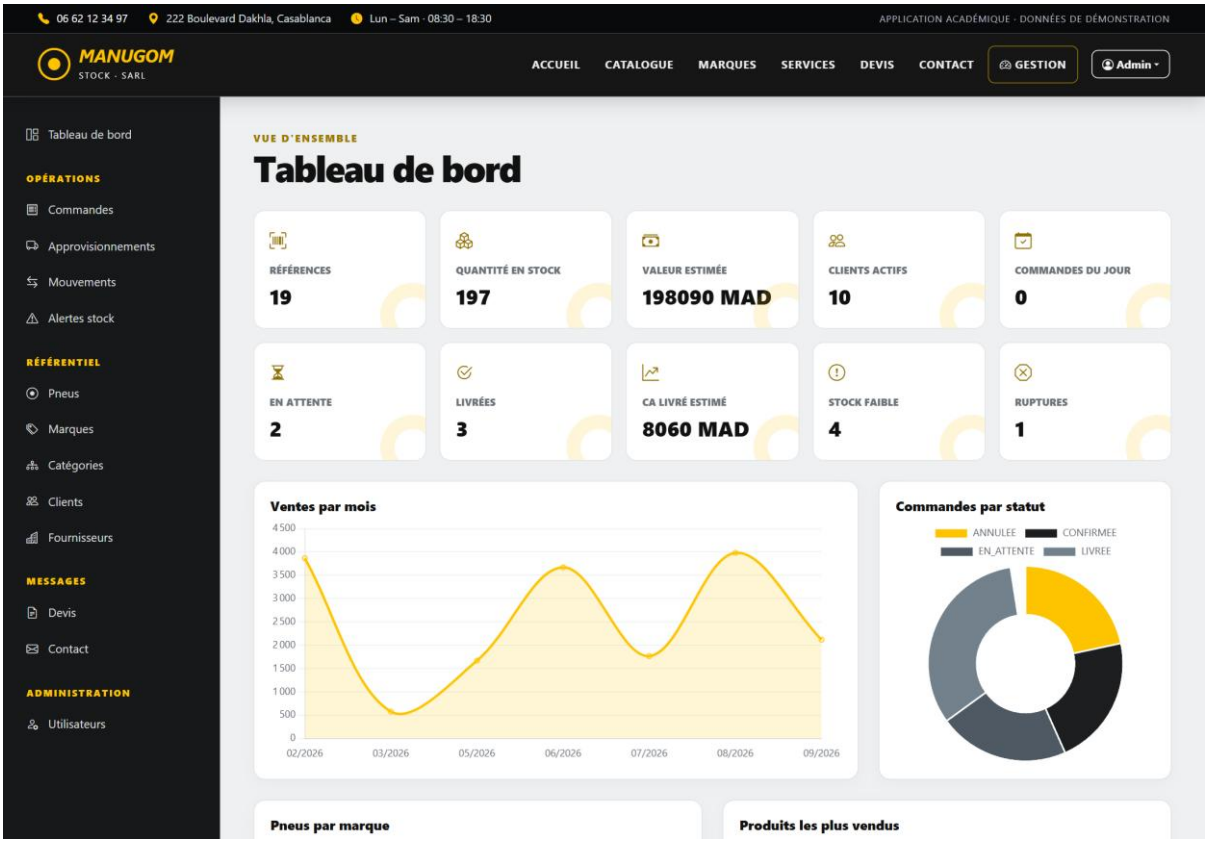


Figure 19 : Indicateurs et ventes du tableau de bord

Interprétation de la figure. Les cartes sont calculées à partir de SQLite : références, quantité, valeur estimée, clients, commandes, chiffre d'affaires livré, stocks faibles et ruptures. Les graphiques présentent les ventes mensuelles et la répartition par statut.

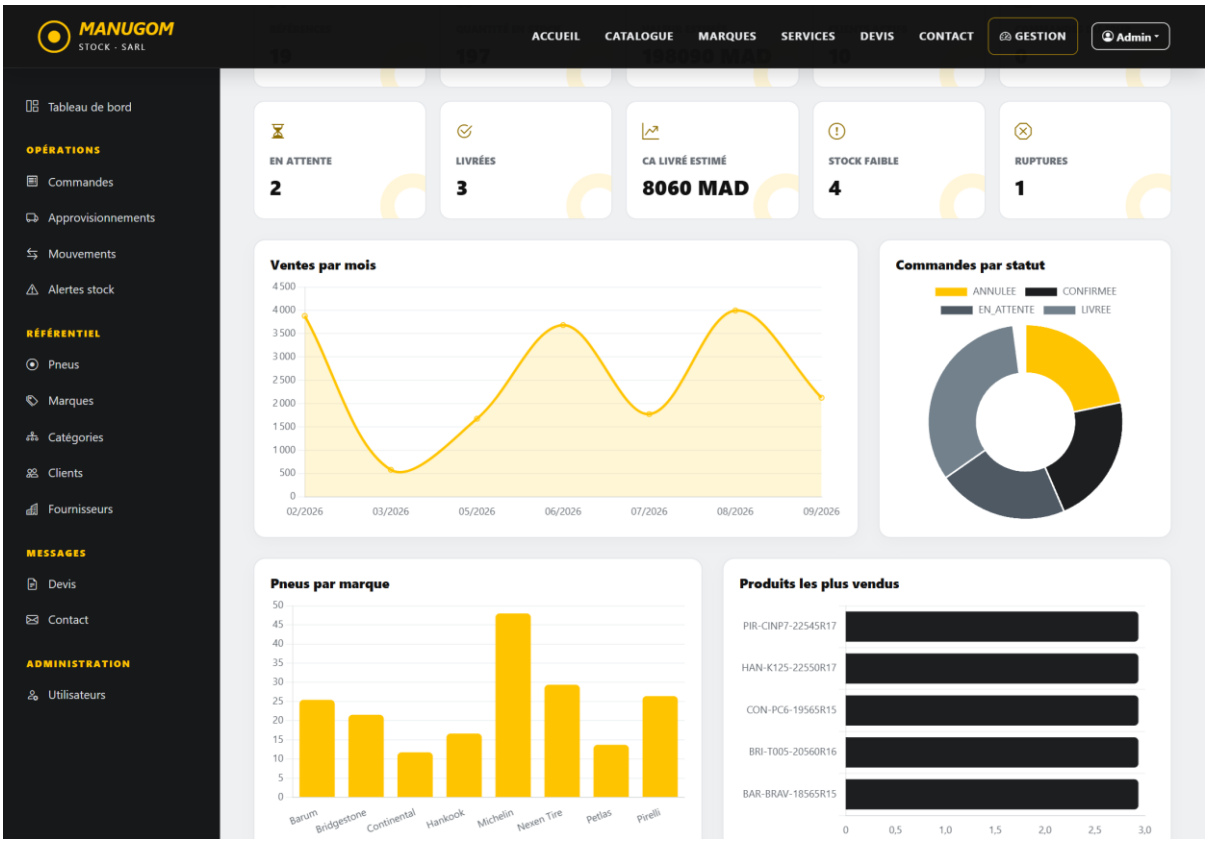


Figure 20 : Analyses complémentaires du stock

Interprétation de la figure. La suite du tableau de bord met en relation les marques, les produits vendus et les mouvements. Il s'agit d'une lecture descriptive du jeu de démonstration. Aucun modèle prédictif ou calcul de réapprovisionnement optimal n'est appliqué.

4.8 Administration

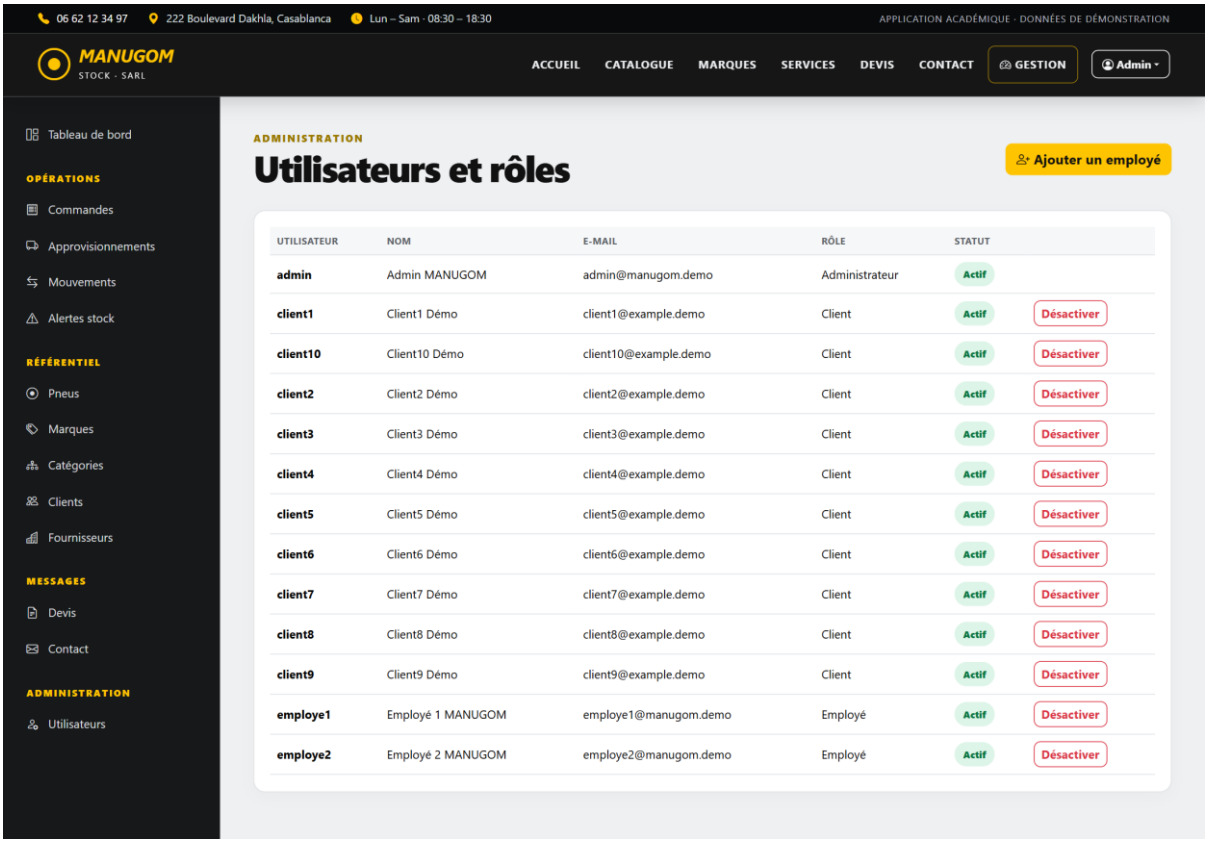


Figure 21 : Gestion des utilisateurs et des rôles

Interprétation de la figure. L'administrateur peut créer un employé et activer ou désactiver les comptes. Les comptes de démonstration sont identifiables par leurs adresses et libellés ; ils doivent être remplacés avant tout usage réel.

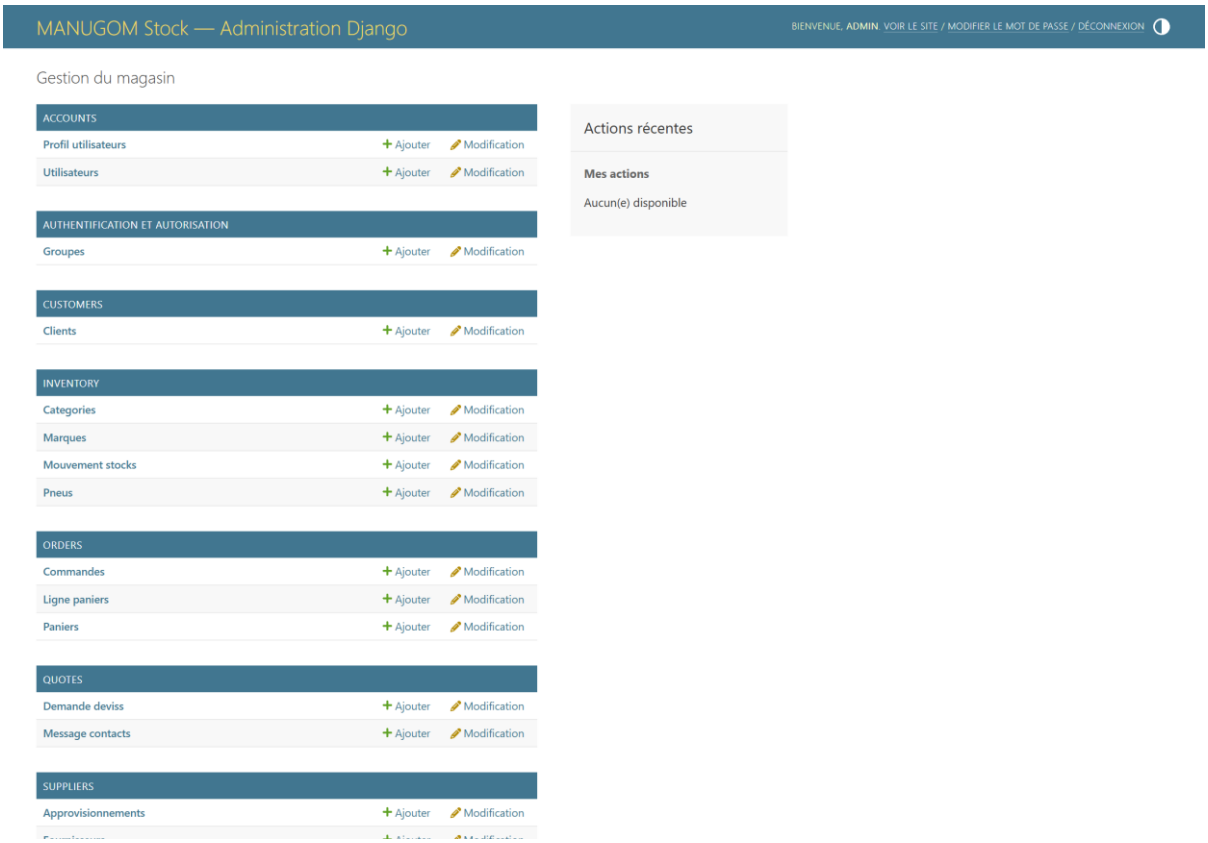


Figure 22 : Administration technique Django

Interprétation de la figure. Django Admin complète l'espace de gestion métier. Il expose les modèles enregistrés aux utilisateurs autorisés, mais ne remplace pas les écrans métier où les transitions et contrôles ont été conçus pour le flux du magasin.

4.9 Cohérence des opérations de stock

ServiceStock relit le pneu dans une transaction, calcule la nouvelle quantité, refuse un résultat négatif, met à jour le stock et crée le mouvement associé. SQLite sérialise les écritures et ne fournit pas ici de verrouillage par ligne. Cette précision est importante : l'appel select_for_update présent dans le code n'a pas l'effet qu'il aurait sur une base serveur compatible.

La confirmation d'une commande appelle le service pour chaque ligne. Les sorties et le changement d'état appartiennent au même bloc transactionnel. Lors d'une annulation autorisée, les retours ne sont créés que si le stock avait été retiré et n'avait pas encore été restauré. Pour un approvisionnement, seul le statut COMMANDE permet la réception et l'opération ne peut être exécutée deux fois.

4.10 Difficultés rencontrées et corrections

La première difficulté concernait la cohérence entre le panier et le stock. La solution retenue distingue la commande en attente de la confirmation par le magasin. La deuxième difficulté portait sur l'historique : les instantanés dans LigneCommande évitent qu'une modification du catalogue transforme une vente passée. La troisième concernait le parcours avant connexion, corrigé pour ne plus rediriger une requête GET vers une vue qui attend POST.

Le périmètre reste centré sur le catalogue, les commandes et le suivi du stock. Les interfaces regroupent les actions utiles à chaque rôle et présentent les résultats des opérations avec des messages explicites. Les captures suivantes documentent les parcours disponibles dans la version réalisée.

Chapitre 5 - Résultats, tests et validation

5.1 Stratégie de test

La suite repose sur les TestCase de Django et crée une base temporaire. Les tests de modèles vérifient les contraintes et les normalisations. Les tests de services couvrent les totaux, les transitions, les sorties, les retours et la réception fournisseur. Les tests de vues examinent les rôles, l'isolation des commandes, les pages publiques et le parcours complet du panier à la commande.

Bertolino [5] rappelle que les tests répondent à des objectifs et niveaux différents. Le résultat 34 sur 34 décrit donc les scénarios exécutés. Il ne correspond ni à un pourcentage de couverture, ni à un audit de sécurité, ni à un essai de charge. L'inspection des captures complète les tests automatiques pour la mise en page visible.

```
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test --verbosity 2
```

5.2 Résultats observés

Tableau 7 : Résultats des contrôles

Contrôle	Résultat	Portée
Suite Django	34 tests réussis, 0 échec	Scénarios de modèles, services, vues et permissions
System check	Aucune anomalie signalée	Configuration analysée par Django
Migrations	Aucun changement détecté	Modèles synchronisés avec les migrations
SQLite	integrity_check = ok	Intégrité structurelle du fichier
Captures	11 pages HTTP 200	Visiteur, client, employé, dashboard et administrateur

Tableau 8 : Jeu de données présent dans SQLite

Entité	Nombre
Utilisateurs	15
Clients	11
Marques	9

Entité	Nombre
Pneus actifs	20
Commandes	11
Approvisionnements	4
Mouvements	41

5.3 Validation par rapport aux objectifs

Tableau 9 : Validation des objectifs

Objectif	État	Évidence
Catalogue structuré et filtrable	Atteint	Captures publiques et tests de rendu
Panier et commandes isolés par client	Atteint	Tests orders et captures client
Stock traçable	Atteint	Mouvements avant et après chaque opération
Approvisionnements contrôlés	Atteint	Réception unique au statut COMMANDE
Administration par rôle	Atteint	Décorateurs et écrans séparés
Prévision ou EOQ	Hors périmètre	Perspective documentée

5.4 Limites actuelles

Le serveur runserver, la clé secrète par défaut et le mode DEBUG conviennent à la démonstration locale. Ils ne forment pas une configuration de production. Les courriels sont écrits dans la console. Aucun paiement, transporteur, API WhatsApp ou service cartographique n'est intégré. Le bouton WhatsApp construit uniquement un lien wa.me.

SQLite simplifie le transport du projet, mais sa concurrence d'écriture est différente de celle d'une base serveur. La suite ne contient ni test multiconnexion, ni campagne de performance, ni test de montée en charge. Une évolution vers PostgreSQL devrait être accompagnée de tests adaptés et d'une stratégie de numérotation qui ne repose pas sur count plus un.

Conclusion générale et perspectives

MANUGOM Stock répond au besoin défini par une application Django qui relie catalogue, comptes, commandes, approvisionnements et mouvements de stock. La séparation des rôles limite les accès, tandis que les services regroupent les transitions qui doivent préserver la cohérence métier. Les diagrammes et les captures montrent la même version que celle soumise aux tests.

Le projet a surtout permis de traiter des problèmes concrets : conserver un prix historique, distinguer une commande en attente d'une confirmation, restaurer une quantité une seule fois et expliquer chaque variation. Les résultats confirment le fonctionnement des scénarios couverts, avec les limites propres à une démonstration locale et à une base SQLite.

Les perspectives les plus utiles concernent un déploiement avec HTTPS et secrets séparés, une base serveur lorsque la concurrence l'exige, des notifications autorisées, la lecture de codes-barres, un inventaire cyclique et des tests de navigateur. Après collecte de données fiables sur la demande et les coûts, une aide au réapprovisionnement pourrait également étudier des quantités économiques, sans confondre un simple seuil avec une optimisation.

Bibliographie

[1] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387. https://doi.org/10.1145/362384.362685

[2] Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053-1058. https://doi.org/10.1145/361598.361623

[3] Härder, T., et Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, 15(4), 287-317. https://doi.org/10.1145/289.291

[4] Sandhu, R., Ferraiolo, D. F., et Kuhn, D. R. (2000). The NIST Model for Role-Based Access Control: Towards a Unified Standard. Proceedings of the Fifth ACM Workshop on Role-Based Access Control, 47-63. https://doi.org/10.1145/344287.344301

[5] Bertolino, A. (2007). Software Testing Research: Achievements, Challenges, Dreams. Future of Software Engineering, 85-103. https://doi.org/10.1109/FOSE.2007.25

[6] Harris, F. W. (1990). How Many Parts to Make at Once. Operations Research, 38(6), 947-950. Réimpression de l'article de 1913. https://doi.org/10.1287/opre.38.6.947

[7] Django Software Foundation. Documentation Django 6.0 et 6.1. https://docs.djangoproject.com/

[8] SQLite Consortium. Isolation In SQLite et documentation générale. https://sqlite.org/isolation.html

[9] Bootstrap Team. Documentation Bootstrap 5.3. https://getbootstrap.com/docs/5.3/

[10] Chart.js Contributors. Documentation Chart.js. https://www.chartjs.org/docs/latest/

[11] StarUML. Documentation officielle. https://docs.staruml.io/

Annexes

Annexe A - Installation et lancement

Depuis PowerShell, placer le terminal dans le dossier manugom-stock puis exécuter les commandes suivantes.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py seed_demo
python manage.py check
python manage.py test
python manage.py runserver
```

Ouvrir ensuite http://127.0.0.1:8000/. L'administration Django se trouve à l'adresse http://127.0.0.1:8000/django-admin/.

Annexe B - Comptes de démonstration

Tableau 10 : Comptes fournis par seed_demo

Rôle	Identifiant	Mot de passe	Accès
Administrateur	admin	Admin@2026!	Gestion et Django Admin
Employé	employe1 ou employe2	Employe@2026!	Espace de gestion
Client	client1 à client10	Client@2026!	Catalogue, panier et commandes

Ces identifiants appartiennent au jeu de démonstration et doivent être remplacés avant un usage réel.

Annexe C - Variables de configuration

```
$env:DJANGO_SECRET_KEY = "votre-cle-longue-et-aleatoire"
$env:DJANGO_DEBUG = "False"
$env:DJANGO_ALLOWED_HOSTS = "votre-domaine.example"
```

Annexe D - Transitions principales

- Commande : EN_ATTENTE vers CONFIRMEE ou ANNULEE.
- Commande confirmée : EN_PREPARATION ou ANNULEE avec retour conditionnel.
- Préparation : PRETE, puis EXPEDIEE ou LIVREE selon le mode de réception.
- Approvisionnement : BROUILLON, COMMANDE, puis RECU une seule fois.
- LIVREE, ANNULEE et RECU sont des états terminaux pour les opérations correspondantes.